Range Slider (Dual Thumb)

- **Problem:** Build a range slider with two thumbs (min and max), like the price filter on Flipkart or Airbnb. The user drags either thumb with mouse, pen or touch, clicks the track to jump the nearest thumb, or uses the keyboard. Values snap to a step, the thumbs never cross and keep a minimum gap, and two number inputs stay in sync with the slider. It must be fully accessible to screen reader and keyboard users.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4) (default, no stack given)
- **Time limit:** 60 minutes (default, no time given)
- **Often asked at:** Flipkart, Airbnb
- **Why it matters:** Pointer events, accessibility

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, component props, pure helper signatures.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Thumbs touching, pointer leaving the track, bad typed input, `min === max`.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	5 min
Hard parts on paper (pixel to value math, crossing, a11y)	5 min
Build: pure `range.js` helpers + quick asserts	8 min
Build: track + two thumbs rendered from `value`	7 min
Build: pointer drag with capture + click on track	10 min

Phase	Time
Build: keyboard + ARIA	7 min
Build: synced number inputs + demo page	6 min
Edge cases, polish, wrap-up	7 min

Tip: think out loud

Say every decision and its reason as you make it: "I keep the value math in pure functions so I can test that thumbs never cross without a browser." The interviewer grades how you think as much as the final code. If you get stuck, say what you are trying and what you would try next.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Two thumbs on one track: **min** and **max**. The part of the track between them is highlighted.
- **Drag** either thumb with mouse, pen or touch. The drag keeps working when the pointer leaves the track.
- **Click** anywhere on the track: the nearest thumb jumps there.
- Values **snap** to a `step` and stay inside `[min, max]`.
- Thumbs **never cross** and keep a **minimum gap** (`minGap`).
- **Keyboard:** arrows move by one step, PageUp/PageDown by a big step, Home/End go to the ends.
- Two **number inputs** (Min, Max) stay in sync both ways.
- Show the current range as formatted text (e.g. `₹2,000 – ₹8,000`).

Non-functional requirements

Area	What we aim for
Performance	One state update per pointer move; no listeners on `window`; no work when the value did not change.
Scalability	Works for any `min / max / step`, including decimals (`step = 0.1`) and huge ranges.
Error handling	Typed text that is empty, negative or too large is clamped or rejected, never crashes. `min === max` and a zero-width track do not divide by zero.
Accessibility	Each thumb is a `role="slider"` with `aria-valuemin/max/now/valuetext`, a label, focus ring and full keyboard support. Touch does not scroll the page while dragging.
Reusability	One controlled `RangeSlider` component, used twice on the demo page (price and discount).
Extensibility	Pure helpers for all math; vertical orientation, RTL or N thumbs only change the helpers and the CSS.

Clarifying questions to ask

1. Should the parent get every value while dragging, or only the final value on release? (Live for the UI, final for an API call.)
2. Can the thumbs touch (`min === max`) or must they keep a minimum gap?
3. What step? Integers only, or decimals like ratings `0.5`?
4. Do we need the typed Min/Max inputs, or only the slider?
5. Any custom marks / ticks or a histogram behind the track (Airbnb style)?
6. Must it work in RTL layouts and vertically?

Assumptions we will build with

- Controlled component: the parent owns `[lo, hi]` and gets every change through `onChange` (it can debounce the API call itself).
- Horizontal, left-to-right only.
- `minGap` is a multiple of `step` (default `0`, so thumbs may touch).
- Number inputs commit on blur or Enter, not on every keystroke.
- Demo: price `₹0–₹10,000`, step `100`, gap `500`; discount `0–100%`, step `1`, no gap.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Drag the min thumb right	Min value follows the pointer, snapped to the step; highlight shrinks from the left
Drag the min thumb past the max thumb	Min stops at `max − minGap`; thumbs never swap
Pointer leaves the track while dragging	Drag keeps going (pointer capture); outside the track the value clamps to `min` / `max`
Release the pointer anywhere	Drag ends; later moves do nothing
Click the track left of the min thumb	Min thumb jumps there and gets focus
Click the track between the thumbs	The closer thumb jumps there (tie goes to min)
Press down where both thumbs overlap, then drag right	The max thumb moves (direction decides which thumb)
Tab	Focus goes to min thumb, then max thumb; focus ring is visible
ArrowRight / ArrowUp	Focused thumb moves `+step` (clamped); page does not scroll
ArrowLeft / ArrowDown	Focused thumb moves `−step`
PageUp / PageDown	Focused thumb moves `±pageStep` (default `10 × step`)
Home / End	Focused thumb goes as far as it can (`min`, or `lo + minGap` for the max thumb)
Type `1234` in Min, blur	Min becomes `1200` (snapped); slider moves
Type `99999` in Min, press Enter	Min clamps to `max thumb − minGap`; input shows the clamped value

Action / input	Expected behavior / output
Clear the Max input, blur	Input goes back to the current max; nothing changes
Drag the slider	Both inputs and the range text update live
Touch drag on a phone	Thumb moves, page does not scroll

3. Design the architecture

Module	Responsibility
`App`	Demo page: owns the `price` and `discount` ranges, renders sliders + inputs, converts typed numbers.
`RangeSlider`	Controlled UI: track, highlight, two thumbs; pointer drag, track clicks, keyboard, ARIA.
`NumberField`	One labelled number input that commits on blur / Enter and shows the value the parent accepted.
`utils/range.js`	Pure math: `clamp`, `snap`, `toPercent`, `pointerToValue`, `setThumb`, `nearestThumb`, `keyToValue`.

```
App  (price: [lo, hi], discount: [lo, hi])
├── <output>        "₹2,000 – ₹8,000"   (derived from price)
├── RangeSlider     props: label, value, onChange, min, max, step, minGap, format
│     ├── track div        pointer events + pointer capture
│     ├── highlight div    left = lo%, right = 100 – hi%
│     └── 2 × thumb div    role="slider", keyboard
├── NumberField × 2 props: label, value, onCommit
└── RangeSlider     (discount, reused)
utils/range.js      used by RangeSlider and App
```

Start simple: one component with two absolutely positioned thumbs and a click handler. Move the math into `utils/range.js` as soon as you write it twice (it is needed by drag, keys and typed input), and split `NumberField` out only when the inputs appear.

4. Design the state / data model

State or field	Purpose
`value: [lo, hi]` (parent)	The only source of truth for the range.
`drag` (ref in slider)	`{ index }` of the thumb being dragged, `index: null` while undecided, `null` when idle. A ref: it changes on every pointer event and never needs a render.
`active` (state in slider)	Which thumb was used last, so it is drawn on top when both overlap.

State or field	Purpose
`trackRef`, `thumbRefs`	DOM nodes to measure the track and move focus.
Input text (DOM)	The half-typed number lives in the uncontrolled `<input>` until blur / Enter.

```
// App (owner)
const [price, setPrice] = useState([2000, 8000]);

// RangeSlider (UI-only state)
const drag = useRef(null);            // { index } | { index: null } | null
const [active, setActive] = useState(1);
```

Senior note. Everything else is derived on render: the thumb positions (`toPercent`), the highlight, `aria-valuemin/max` of each thumb (they depend on the other thumb), and the range text. The parent owns `value` because the parent is the one that filters products with it; the slider owns only what is about the gesture (`drag` , `active`). The drag info is a ref, not state, because no pixel depends on it.

5. Design the data structures and contracts

Example data

```
const PRICE = { min: 0, max: 10000, step: 100, minGap: 500 };
const value = [2000, 8000];             // always sorted: value[0] <= value[1]
```

A tuple, not `{ from, to }` : index `0` / `1` is the thumb index, so the same code handles both thumbs (`value[index]` , `setThumb(value, index, next)`). No ids needed: there are always exactly two thumbs in a fixed order.

`RangeSlider` props

Prop	Type	Default	Meaning
`label`	string	required	`"Price"` → thumbs are named "Minimum price" / "Maximum price"
`value`	`[number, number]`	required	Current range
`onChange`	`(next: [number, number]) => void`	required	Called only when the value really changes
`min` , `max`	number	`0` , `100`	Bounds
`step`	number	`1`	Snap grid, anchored at `min`
`pageStep`	number	`10 × step`	PageUp / PageDown jump
`minGap`	number	`0`	Smallest allowed `hi − lo`

Prop	Type	Default	Meaning
`format`	`(v) => string`	`String`	Text for `aria-valuetext` (e.g. `₹2,000`)

Pure helper contracts

```
snap(14, 0, 10)                                  // 10
pointerToValue(200, { left: 100, width: 200 }, { min: 0, max: 100, step: 10 })   // 50
setThumb([20, 80], 0, 95, { min: 0, max: 100, minGap: 20 })                      // [60, 80]
nearestThumb([20, 80], 70)                       // 1
keyToValue('PageUp', 20, { min: 0, max: 100, step: 10, pageStep: 30 })          // 50 (unclamped)
```

There is no API in this problem. In a real filter the parent would debounce `value` and call `GET /products?minPrice=2000&maxPrice=8000`.

6. Trace the data flow

Drag the min thumb from ₹2,000 to the right, past the max thumb

```
pointerdown on track at x
   ↓  setPointerCapture(pointerId)       all later moves come to the track, even off it
   ↓  pointerToValue(x)  →  2000         nearestThumb([2000, 8000], 2000) → 0
   ↓  drag.current = { index: 0 }, setActive(0), focus min thumb
pointermove (x grows)
   ↓  pointerToValue(x)  →  9400         rect → ratio → value → snapped to 100
   ↓  setThumb([2000, 8000], 0, 9400)    clamp(9400, 0, 8000 − 500) → [7500, 8000]
   ↓  changed? yes  →  onChange([7500, 8000])
parent: setPrice([7500, 8000])
   ↓  re-render
RangeSlider: thumb left = 75%, highlight 75%–80%, aria-valuenow = 7500, max thumb aria-valuemin = 8000
App: "₹7,500 – ₹8,000", Min input effect writes 7500
pointerup / pointercancel / lostpointercapture
   ↓  drag.current = null
```

7. Important technical challenges

Challenge 1: A drag that never loses the thumb

Problem. With `onMouseMove` on the thumb, a fast flick moves the pointer off the 20px thumb and the drag stops. Putting `mousemove` on `window` works but needs manual add/remove and leaks if you forget. On touch, the browser starts scrolling the page and fires `pointercancel`, so the thumb freezes.

Solution. Use **Pointer Events** (one code path for mouse, pen, touch) on the **track**, and call `setPointerCapture` on `pointerdown`: until release, every move for that pointer goes to the track wherever the finger is. Add `touch-action: none` (`touch-none`) so the browser does not pan. End the drag on `pointerup`, `pointercancel` and `lostpointercapture`.

```
const onPointerDown = (e) => {
  if (e.button !== 0) return;
  e.preventDefault();                        // no text selection
  e.currentTarget.setPointerCapture(e.pointerId);
  ...
};
<div ref={trackRef} onPointerDown={onPointerDown} onPointerMove={onPointerMove}
     onPointerUp={endDrag} onPointerCancel={endDrag} onLostPointerCapture={endDrag}
     className="relative h-6 touch-none select-none" />
```

Handlers are recreated on each render, so they read the latest `value`. During a drag only the dragged thumb changes and we read just the other thumb from `value`, so a move that arrives before React re-renders cannot corrupt anything.

Challenge 2: Pixel → value math (snapping, floats, bounds)

Problem. `clientX` is a pixel; the value must be inside `[min, max]` and on the step grid. Three traps: the pointer goes outside the track (ratio < 0 or > 1), floats (`0.1 + 0.2 = 0.30000000000000004` shows up in the UI and in `aria-valuenow`), and the grid must start at `min` (with `min = 5, step = 10` valid values are 5, 15, 25…, not 10, 20).

Solution. Clamp the ratio, map it to the range, then snap relative to `min` and round to the step's decimal places.

```
export function snap(value, min, step) {
  const decimals = (String(step).split('.')[1] || '').length;
  return Number((Math.round((value - min) / step) * step + min).toFixed(decimals));
}

export function pointerToValue(clientX, rect, { min, max, step }) {
  const ratio = rect.width ? clamp((clientX - rect.left) / rect.width, 0, 1) : 0;
  return snap(min + ratio * (max - min), min, step);
}
```

Measure the rect on each event (`getBoundingClientRect`), not once: the page can scroll or resize mid-drag.

Challenge 3: Two thumbs that never cross, and overlap

Problem. (a) Dragging min past max would invert the range. (b) Clicking the track: which thumb moves? (c) With `minGap = 0` both thumbs can sit on the same spot; if we always pick min and both are at `0`, the user can never drag them apart to the right, because min cannot pass max. (d) The thumb drawn last always covers the other.

Solution.

- One function moves one thumb and clamps it against the other thumb, so every input (drag, keys, typing) gets the rule for free:

```
export function setThumb([lo, hi], index, next, { min, max, minGap = 0 }) {
  return index === 0 ? [clamp(next, min, hi - minGap), hi] : [lo, clamp(next, lo + minGap, max)];
}
```

- A click picks the nearest thumb; outside the range goes to that side's thumb.
- When both thumbs overlap and the press lands on them, wait for the first move: moving right picks max, left picks min.

```
if (lo === hi && next === lo) { drag.current = { index: null }; return; }   // pointerdown
if (drag.current.index === null) {                                          // pointermove
  if (next === lo) return;
  grab(next > lo ? 1 : 0);
}
```

- The last-used thumb gets `z-20`, the other `z-10`.

Challenge 4: Accessibility (two sliders, not one)

Problem. A `div` thumb is invisible to screen readers and the keyboard. Two native `<input type="range">` stacked on top of each other is a common hack, but only the top one gets pointer events and styling them is painful.

Solution. Follow the WAI-ARIA multi-thumb slider pattern: each thumb is `role="slider"`, `tabIndex={0}`, with a name and **dynamic bounds** (the other thumb limits it), plus a human `aria-valuetext`. Arrow keys call `preventDefault` so the page does not scroll.

```
<div role="slider" tabIndex={0}
  aria-label={`${THUMB_NAMES[i]} ${label.toLowerCase()}`}     // "Minimum price"
  aria-valuemin={i === 0 ? min : lo + minGap}
  aria-valuemax={i === 0 ? hi - minGap : max}
  aria-valuenow={v}
  aria-valuetext={format(v)}                                  // "₹2,000"
  onKeyDown={onKeyDown(i)} />
```

Clicking the track moves focus to the thumb that moved, so a mouse user can continue with the arrow keys.

8. Implement in the correct order

#	Step	Working result
1	Vite + Tailwind, `App` with `useState([2000, 8000])`	Page renders the numbers
2	`utils/range.js`: `clamp`, `snap`, `toPercent` + `node` asserts	Math proven before UI
3	`RangeSlider` renders track, highlight, two thumbs from `value`	Static slider in the right place

#	Step	Working result
4	Click on track: `pointerToValue` + `nearestThumb` + `setThumb`	Clicking moves the nearest thumb
5	Drag: `setPointerCapture`, `pointermove`, end on up/cancel/lost	Smooth drag, also off the track
6	`touch-none`, `select-none`, `button !== 0` guard	Works on phones, no text selection
7	Overlapping thumbs: undecided drag + `active` z-index	Stacked thumbs can be pulled apart
8	Keyboard: `keyToValue` + `preventDefault`	Arrows, PageUp/Down, Home/End
9	ARIA: role, label, dynamic min/max, valuetext, focus ring	Screen reader announces "Minimum price, ₹2,000"
10	`NumberField` with commit on blur / Enter	Typing syncs the slider and back
11	Reuse for discount; `minGap`, `format` props	Component proven reusable
12	Edge-case pass with the table below	Done

9. Cover all states and edge cases

State / edge case	Behavior
Initial	Thumbs at the given `value`; max thumb on top
Idle / dragging	`drag.current` is `null` / `{ index }`; only dragging reacts to `pointermove`
Thumbs touching (`minGap = 0`)	Both at the same spot; first move direction picks the thumb
Thumbs at the minimum gap	Each one stops at the other ± `minGap`; `aria-valuemin/max` say so
Both at `min` or both at `max`	Still separable (direction rule)
Pointer leaves the track / window	Capture keeps the drag; value clamps to the ends
Pointer cancel (touch scroll, system gesture)	Drag ends, value stays where it was
Right click / middle click	Ignored
Move that does not change the snapped value	No `onChange`, no re-render
Decimal step (`0.1`)	No `0.30000000000000004`
`min === max` / zero-width track	`toPercent` and `pointerToValue` return safe values, no `NaN`
Typed empty / non-number	Input resets to the current value
Typed off-grid (`1234`)	Snapped to `1200`
Typed beyond bounds or past the other thumb	Clamped; the input shows the clamped value even if state did not change

State / edge case	Behavior
Arrow key at the end	Value stays; no `onChange`
Arrow keys / PageUp	Page does not scroll
Page scrolled or resized mid-drag	Rect measured on every event
Value changed from outside (reset button)	Slider and inputs follow (controlled + effect)

10. Final structure

```
_80_range_slider_dual_thumb/
├── index.html
├── package.json
├── vite.config.js
├── check.mjs                    node self-check for utils/range.js
└── src/
    ├── main.jsx
    ├── index.css                @import "tailwindcss"
    ├── App.jsx                  demo: price + discount filters
    ├── components/
    │   ├── RangeSlider.jsx      track, thumbs, pointer + keyboard + ARIA
    │   └── NumberField.jsx      number input that commits on blur / Enter
    └── utils/
        └── range.js             pure value math
```

11. Full working code

Create the project with `npm create vite@latest` (React, JavaScript), then `npm i -D tailwindcss @tailwindcss/vite`.

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

`index.html`

```
<!doctype html>
<html lang="en">
  <head>
```

```
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Range Slider</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/index.css

```
@import "tailwindcss";
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/utils/range.js

```
export const clamp = (v, lo, hi) => Math.min(hi, Math.max(lo, v));

// Snap to the step grid anchored at min; toFixed removes float noise like 0.30000000000000004
export function snap(value, min, step) {
  const decimals = (String(step).split('.')[1] || '').length;
  return Number((Math.round((value - min) / step) * step + min).toFixed(decimals));
}

export const toPercent = (value, min, max) => (max === min ? 0 : ((value - min) / (max - min)) * 100);

// Pixel position on the track -> snapped value inside [min, max]
export function pointerToValue(clientX, rect, { min, max, step }) {
  const ratio = rect.width ? clamp((clientX - rect.left) / rect.width, 0, 1) : 0;
  return snap(min + ratio * (max - min), min, step);
}

// Move one thumb; it can never cross the other or get closer than minGap
```

```
export function setThumb([lo, hi], index, next, { min, max, minGap = 0 }) {
  return index === 0 ? [clamp(next, min, hi - minGap), hi] : [lo, clamp(next, lo + minGap, max)];
}

export function nearestThumb([lo, hi], value) {
  if (value <= lo) return 0;
  if (value >= hi) return 1;
  return value - lo <= hi - value ? 0 : 1;
}

// Keyboard -> new value (unclamped), or null when the key is not ours
export function keyToValue(key, value, { min, max, step, pageStep }) {
  const by = (delta) => snap(value + delta, min, step);
  switch (key) {
    case 'ArrowRight':
    case 'ArrowUp':
      return by(step);
    case 'ArrowLeft':
    case 'ArrowDown':
      return by(-step);
    case 'PageUp':
      return by(pageStep);
    case 'PageDown':
      return by(-pageStep);
    case 'Home':
      return min;
    case 'End':
      return max;
    default:
      return null;
  }
}
```

src/components/RangeSlider.jsx

```
import { useRef, useState } from 'react';
import { keyToValue, nearestThumb, pointerToValue, setThumb, toPercent } from '../utils/range';

const THUMB_NAMES = ['Minimum', 'Maximum'];

export default function RangeSlider({
  label,
  value,
  onChange,
  min = 0,
  max = 100,
  step = 1,
```

```
  pageStep = step * 10,
  minGap = 0,
  format = String,
}) {
  const trackRef = useRef(null);
  const thumbRefs = [useRef(null), useRef(null)];
  const drag = useRef(null); // { index } while dragging; index is null until the direction is known
  const [active, setActive] = useState(1); // last-used thumb is drawn on top
  const opts = { min, max, step, minGap };
  const [lo, hi] = value;

  const update = (index, next) => {
    const nextValue = setThumb(value, index, next, opts);
    if (nextValue[0] !== lo || nextValue[1] !== hi) onChange(nextValue);
  };

  const grab = (index) => {
    drag.current = { index };
    setActive(index);
    thumbRefs[index].current.focus();
  };

  const valueAt = (e) => pointerToValue(e.clientX, trackRef.current.getBoundingClientRect(), opts);

  const onPointerDown = (e) => {
    if (e.button !== 0) return;
    e.preventDefault(); // no text selection; we move focus ourselves
    e.currentTarget.setPointerCapture(e.pointerId);
    const next = valueAt(e);
    // Both thumbs on the same spot: wait for the first move to know which one the user wants
    if (lo === hi && next === lo) {
      drag.current = { index: null };
      return;
    }
    const index = nearestThumb(value, next);
    grab(index);
    update(index, next);
  };

  const onPointerMove = (e) => {
    if (!drag.current) return;
    const next = valueAt(e);
    if (drag.current.index === null) {
      if (next === lo) return;
      grab(next > lo ? 1 : 0);
    }
    update(drag.current.index, next);
```

```
};

const endDrag = () => {
  drag.current = null;
};

const onKeyDown = (index) => (e) => {
  const next = keyToValue(e.key, value[index], { ...opts, pageStep });
  if (next === null) return;
  e.preventDefault(); // arrows/PageUp must not scroll the page
  setActive(index);
  update(index, next);
};

const pct = [toPercent(lo, min, max), toPercent(hi, min, max)];

return (
  <div className="px-2.5 py-2">
    <div
      ref={trackRef}
      onPointerDown={onPointerDown}
      onPointerMove={onPointerMove}
      onPointerUp={endDrag}
      onPointerCancel={endDrag}
      onLostPointerCapture={endDrag}
      className="relative h-6 cursor-pointer touch-none select-none"
    >
      <div className="absolute inset-x-0 top-1/2 h-1.5 -translate-y-1/2 rounded-full bg-slate-200" />
      <div
        className="absolute top-1/2 h-1.5 -translate-y-1/2 rounded-full bg-blue-600"
        style={{ left: `${pct[0]}%`, right: `${100 - pct[1]}%` }}
      />
      {value.map((v, i) => (
        <div
          key={THUMB_NAMES[i]}
          ref={thumbRefs[i]}
          role="slider"
          tabIndex={0}
          aria-label={`${THUMB_NAMES[i]} ${label.toLowerCase()}`}
          aria-orientation="horizontal"
          aria-valuemin={i === 0 ? min : lo + minGap}
          aria-valuemax={i === 0 ? hi - minGap : max}
          aria-valuenow={v}
          aria-valuetext={format(v)}
          onKeyDown={onKeyDown(i)}
          style={{ left: `${pct[i]}%` }}
          className={`absolute top-1/2 size-5 -translate-x-1/2 -translate-y-1/2 rounded-full border-2
```

```
border-blue-600 bg-white shadow outline-none focus-visible:ring-4 focus-visible:ring-blue-300 ${
              active === i ? 'z-20' : 'z-10'
            }`}
          />
        ))}
      </div>
    </div>
  );
}
```

src/components/NumberField.jsx

```
import { useEffect, useRef } from 'react';

// Uncontrolled so half-typed numbers are not clamped on every keystroke; commits on blur / Enter
export default function NumberField({ label, value, onCommit }) {
  const ref = useRef(null);

  useEffect(() => {
    ref.current.value = value; // the slider moved: show the new number
  }, [value]);

  const commit = () => {
    ref.current.value = onCommit(ref.current.valueAsNumber);
  };

  return (
    <label className="flex-1 text-sm text-slate-600">
      {label}
      <input
        ref={ref}
        type="number"
        inputMode="numeric"
        defaultValue={value}
        onBlur={commit}
        onKeyDown={(e) => e.key === 'Enter' && commit()}
        className="mt-1 w-full rounded-md border border-slate-300 px-3 py-2 text-slate-900"
      />
    </label>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import RangeSlider from './components/RangeSlider';
```

```
import NumberField from './components/NumberField';
import { setThumb, snap } from './utils/range';

const PRICE = { min: 0, max: 10000, step: 100, minGap: 500 };
const inr = (n) => `₹${n.toLocaleString('en-IN')}`;

export default function App() {
  const [price, setPrice] = useState([2000, 8000]);
  const [discount, setDiscount] = useState([10, 60]);

  // Typed value -> snapped + clamped like a drag; returns what the input should show
  const commitPrice = (index) => (n) => {
    if (Number.isNaN(n)) return price[index];
    const next = setThumb(price, index, snap(n, PRICE.min, PRICE.step), PRICE);
    setPrice(next);
    return next[index];
  };

  return (
    <main className="min-h-screen bg-slate-50 p-4 sm:p-8">
      <section className="mx-auto max-w-md space-y-8 rounded-xl bg-white p-6 shadow">
        <div className="space-y-3">
          <div className="flex items-baseline justify-between">
            <h2 className="font-semibold text-slate-900">Price</h2>
            <output className="text-sm text-slate-600">
              {inr(price[0])} – {inr(price[1])}
            </output>
          </div>
          <RangeSlider label="Price" value={price} onChange={setPrice} format={inr} {...PRICE} />
          <div className="flex gap-3">
            <NumberField label="Min" value={price[0]} onCommit={commitPrice(0)} />
            <NumberField label="Max" value={price[1]} onCommit={commitPrice(1)} />
          </div>
        </div>

        <div className="space-y-3">
          <div className="flex items-baseline justify-between">
            <h2 className="font-semibold text-slate-900">Discount</h2>
            <output className="text-sm text-slate-600">
              {discount[0]}% – {discount[1]}%
            </output>
          </div>
          <RangeSlider label="Discount" value={discount} onChange={setDiscount} format={(v) => `${v}%`}
/>
        </div>
      </section>
    </main>
```

```
  );
}
```

`check.mjs` **(run with** `node check.mjs` **)**

```
import assert from 'node:assert/strict';
import { keyToValue, nearestThumb, pointerToValue, setThumb, snap, toPercent } from
'./app/src/utils/range.js';

const opts = { min: 0, max: 100, step: 10, minGap: 20 };
const rect = { left: 100, width: 200 };

// snapping: grid anchored at min, no float noise
assert.equal(snap(0.1 + 0.2, 0, 0.1), 0.3);
assert.equal(snap(14, 0, 10), 10);
assert.equal(snap(15, 0, 10), 20);
assert.equal(snap(7, 5, 10), 5);

// pixel -> value, clamped to the track; zero-width track does not divide by zero
assert.equal(pointerToValue(200, rect, opts), 50);
assert.equal(pointerToValue(50, rect, opts), 0);
assert.equal(pointerToValue(999, rect, opts), 100);
assert.equal(pointerToValue(150, { left: 0, width: 0 }, opts), 0);
assert.equal(toPercent(5, 5, 5), 0);

// thumbs never cross and keep minGap
assert.deepEqual(setThumb([20, 80], 0, 95, opts), [60, 80]);
assert.deepEqual(setThumb([20, 80], 1, 0, opts), [20, 40]);
assert.deepEqual(setThumb([20, 80], 0, -50, opts), [0, 80]);
assert.deepEqual(setThumb([20, 80], 1, 500, opts), [20, 100]);

// nearest thumb picks the closer one, ties go left, outside goes to that side
assert.equal(nearestThumb([20, 80], 40), 0);
assert.equal(nearestThumb([20, 80], 70), 1);
assert.equal(nearestThumb([20, 80], 50), 0);
assert.equal(nearestThumb([20, 80], 90), 1);

// keyboard
const k = { ...opts, pageStep: 30 };
assert.equal(keyToValue('ArrowRight', 20, k), 30);
assert.equal(keyToValue('ArrowDown', 20, k), 10);
assert.equal(keyToValue('PageUp', 20, k), 50);
assert.equal(keyToValue('Home', 50, k), 0);
assert.equal(keyToValue('End', 50, k), 100);
assert.equal(keyToValue('a', 50, k), null);
assert.equal(keyToValue('ArrowUp', 0.2, { min: 0, max: 1, step: 0.1, pageStep: 1 }), 0.3);
```

```
console.log('range.js self-check passed');
```

12. Wrap-up

Edge cases handled

- Thumbs never cross, keep `minGap`, and can be separated even when stacked at either end.
- Drag survives leaving the track; ends on up, cancel and lost capture; touch does not scroll.
- Snap grid anchored at `min`, no float noise, safe with `min === max` and a zero-width track.
- Keyboard: arrows, PageUp/Down, Home/End, no page scroll, focus follows track clicks.
- Typed input: empty, off-grid, out-of-range and past-the-other-thumb values are fixed, not crashed on.

Trade-offs to mention

- **Custom `div` thumbs vs two native `<input type="range">`.** Native inputs give a11y and keyboard for free, but stacking two needs `pointer-events` hacks, the overlap problem gets worse, and cross-browser thumb styling is painful. Custom thumbs cost us the ARIA work but give full control.
- **Controlled only.** The parent must hold state. Simple and predictable; an uncontrolled `defaultValue` mode would add code nobody asked for.
- **`onChange` on every move.** Great for live UI; an API call should be debounced by the parent, or we add an `onChangeEnd` fired on `pointerup` / `keyup`.
- **Pointer capture on the track vs `window` listeners.** Capture needs no cleanup and works for touch. It does not exist in very old browsers (fine for any modern target).
- **Percent positioning.** `left: x%` re-flows nothing else and scales with the container for free; for 60fps on heavy pages `transform: translateX` is cheaper.
- **Uncontrolled number inputs.** Avoids clamping half-typed text on each keystroke; the cost is a small effect to write the value back.

What to add with more time

- `onChangeEnd` / commit callback, and a debounced fetch of filtered results in the demo.
- Vertical orientation and RTL (flip the ratio; `aria-orientation="vertical"`).
- Tick marks, labels, and an Airbnb style histogram behind the track.
- Tooltip bubble above the thumb while dragging.
- Disabled and read-only states; form integration with hidden inputs (`name="minPrice"`).
- React Testing Library + `user-event` tests for keys and inputs; Playwright for real drags on desktop and touch.
- N thumbs (generalize `setThumb` to clamp between neighbours).

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use two `<input type="range">` on top of each other?

It is a fine 10-minute answer and you get keyboard and a11y for free. The problems: only the top input receives pointer events (you need `pointer-events: none` on the track and `auto` on the thumbs), when

thumbs overlap the wrong one is grabbed, crossing still has to be prevented in JS, and thumb styling differs per browser. A custom slider fixes all of that for ~100 lines.

Q2. The parent calls an API with the range. What changes?

Nothing in the slider. The parent keeps `value` for the UI and debounces the request (300 ms), or we add an `onChangeEnd` that fires on `pointerup` and key release so the API is called once per gesture. Cancel the old request with `AbortController` so a slow response cannot overwrite a newer one.

Q3. What if they want ticks or labels under the track?

Ticks are derived: `for (v = min; v <= max; v += tickStep)` → render at `toPercent(v)`. Clicking a label calls the same `nearestThumb` + `setThumb` path. Nothing in the state model changes.

State design

Q4. Why is `value` a tuple and not `{ min, max }`?

The thumb index is the array index, so one handler works for both thumbs: `value[index]`, `setThumb(value, index, next)`. With an object you write `index === 0 ? 'min' : 'max'` everywhere. It also generalizes to N thumbs.

Q5. Why is the drag info a ref and not state?

It changes on every `pointerdown` / `pointerup` and no pixel depends on it. As state it would cause extra renders and, worse, handlers could read an old value between two fast events. The `active` thumb *is* state because the z-index depends on it.

Q6. Controlled or uncontrolled? Why?

Controlled: the filter value belongs to the page (it drives the product list, the URL, the inputs). The slider only owns gesture state. This also lets a "Reset filters" button move the slider by just setting state.

Challenge 1: pointer events

Q7. Why `setPointerCapture` instead of listening on `window`?

Capture routes every event of that pointer to the track until release, so it keeps working off the track, off the window, and on touch, with no add/remove and no leak. Window listeners need cleanup on unmount and on every exit path, and touch still needs `touch-action`.

Q8. Why `touch-action: none`? What happens without it?

On touch the browser decides on the first move whether it is a scroll. If it is, it sends `pointercancel` and scrolls the page: the thumb freezes. `touch-action: none` on the track tells it not to pan there. Only the track gets it, so the rest of the page scrolls normally.

Q9. What does `e.preventDefault()` on `pointerdown` do here?

It stops the compatibility mouse events, so the browser does not start a text selection or move focus to the track. We then focus the right thumb ourselves. `select-none` covers the rest of the selection cases.

Challenge 2: value math

Q10. The step is 0.1 and the UI shows 0.30000000000000004. Fix?

Round to the step's number of decimals after snapping: `Number(x.toFixed(decimals))`. Do it in one place (`snap`) so drag, keys and typing are all clean. Alternatively store integers (tenths) and divide only for display.

Q11. Why anchor the grid at `min`?

With `min = 5, step = 10`, `Math.round(v / step) * step` gives 0, 10, 20: values the user cannot select and `min` itself is off-grid. Snapping `(v - min)` and adding `min` back gives 5, 15, 25, matching the native `<input type="range">`.

Q12. What if the range is 0 to 1,000,000 with step 1 on a 300px track?

Each pixel is ~3,333 units, so the mouse cannot hit every value. That is fine: the keyboard (and the number inputs) give exact values, PageUp/Down give big jumps. If needed, a non-linear (log) scale maps pixels to values: change only `pointerToValue` and `toPercent`.

Challenge 3: crossing and overlap

Q13. Why not swap the thumbs when min passes max (some libraries do)?

Swapping means the thumb under your finger suddenly becomes the "max" thumb, focus and ARIA labels jump, and the screen reader announces the wrong thing. Clamping (`setThumb`) keeps identity stable and is one line. Swapping is a valid product choice, but you must then also swap focus.

Q14. Both thumbs sit at 0. The user drags. What happens?

If we always grabbed the min thumb it could never move right (it cannot pass max): a stuck slider. We wait for the first `pointermove`: right means max, left means min. The same works at the top end.

Q15. Why does the last-used thumb get a higher `z-index`?

When thumbs overlap, the one on top gets the visual focus and is the one you see moving. Without it, the max thumb (rendered last) always covers the min thumb, even right after you moved min.

Challenge 4: accessibility

Q16. Why are `aria-valuemin` / `aria-valuemax` different per thumb?

The real limit of the min thumb is the max thumb minus the gap, not the slider's `max`. Screen readers use these to tell the user how far they can go, and some announce percentages from them. The WAI-ARIA multi-thumb slider pattern asks for exactly this.

Q17. Why `aria-valuetext`?

`aria-valuenow` is a raw number ("2000"). `aria-valuetext` gives "₹2,000", or "2 stars", or "10:30 AM". The `format` prop makes the same component say the right thing in every use.

Q18. How would a screen reader user use it?

Tab to "Minimum price, slider, ₹2,000", arrow keys change it by one step, PageUp/Down by ten steps, Home/End to the limits, Tab to "Maximum price". The visible `focus-visible` ring shows keyboard users where they are.

Implementation

Q19. Why does the number input commit on blur and not on every keystroke?

Typing `5000` passes through `5`, `50`, `500`. If each keystroke clamps (e.g. to the 500 gap), the user cannot type the number they want. So the input is uncontrolled while typing and only commits on blur / Enter. The value we actually accepted is written back to the input.

Q20. Two key presses arrive before React re-renders. Is the second one lost?

Real key events arrive in separate tasks, and React flushes discrete events like `keydown` synchronously, so the handler always sees the new `value`. If you need to be safe against batched events too, keep `value` in a ref updated during render and compute from it, or use a functional updater in the parent.

Performance and scale

Q21. What if this slider is on a page with 5,000 products filtered on every move?

The slider is cheap; the list is not. Keep the slider state fast and pass a debounced or `useDeferredValue(value)` to the filter, so the thumb stays at 60fps while the list updates when React has time. Virtualize the list if it is long.

Q22. Should thumbs use `left` or `transform`?

`left: x%` is simple and scales with the container without measuring. `transform: translateX()` skips layout and is cheaper per frame; you would need the track width in px (or a CSS variable with `calc`). For two thumbs `left` is fine; for an animation-heavy page switch to `transform`.

Trade-offs and twists

Q23. How would you support RTL and vertical sliders?

RTL: `ratio = (rect.right - clientX) / rect.width` and position with `right: x%`; also swap ArrowLeft / ArrowRight so the key moves the thumb the way it points. Vertical: use `clientY` and the rect height, invert because up means more, set `aria-orientation="vertical"`. All of it is in `pointerToValue` and the styles.

Q24. How would you test it?

Pure helpers with `node` asserts (done in `check.mjs`). Component: React Testing Library + `user-event` for Tab, arrows and typed inputs, asserting `aria-valuenow`. Drag: Playwright with `mouse.down / move / up` and touch emulation, because jsdom has no layout (every rect is 0).

Q25. How would you make it work with N thumbs (e.g. a 3-point pricing tier)?

`value` becomes an array of N sorted numbers. `setThumb` clamps between neighbours `value[i - 1] + gap` and `value[i + 1] - gap`. `nearestThumb` picks the closest index, and the overlap rule picks by direction

among the stacked ones. Render thumbs with `value.map`; the highlight sits between the first and last.